UNIVERSITÉ : .....

ÉTABLISSEMENT / DÉPARTEMENT : .....

RAPPORT DE TRAVAUX PRATIQUES

TP 3 : SYSTÈME DE GESTION D'UNE RÉSIDENCE UNIVERSITAIRE

Module : Modélisation UML et programmation orientée objet

Présenté par :	.....
Filière / Niveau :	.....
Enseignant :	.....
Année académique :	.....

Année académique : 2026–2027

1. Introduction

La gestion d'une résidence universitaire nécessite le suivi des chambres, des étudiants, des demandes d'hébergement et des périodes d'occupation. Lorsque ces opérations sont effectuées manuellement, le responsable peut rencontrer des difficultés pour connaître les places disponibles, éviter les doubles affectations et retrouver l'historique des occupants.

Ce travail pratique porte sur la conception d'un système informatique destiné à faciliter la gestion d'une résidence universitaire. La modélisation est réalisée à l'aide du langage UML.

2. Objectifs du système

L'objectif général est de proposer une application permettant au responsable de gérer les hébergements de manière organisée et de suivre les occupations des chambres.

• Enregistrer les demandes d'hébergement des étudiants.

• Attribuer une chambre à un étudiant lorsque des places sont disponibles.

• Consulter les chambres libres et occupées.

• Enregistrer les départs et clôturer les périodes d'occupation.

• Consulter l'historique des attributions.

3. Présentation générale du fonctionnement

L'étudiant soumet une demande d'hébergement. Le responsable consulte cette demande et vérifie les disponibilités de la résidence. Si une chambre dispose d'une place suffisante et qu'aucune attribution incompatible n'existe, le responsable procède à l'affectation. L'attribution est enregistrée avec une date de début. Au départ de l'étudiant, une date de fin est ajoutée afin de libérer la place et de conserver l'historique.

Le système distingue ainsi la demande d'hébergement, qui exprime le besoin de l'étudiant, et l'attribution, qui matérialise l'affectation effective d'une chambre.

4. Classes principales

Classe	**Rôle**
Etudiant	Contient les informations permettant d'identifier un étudiant.
Residence	Représente la résidence universitaire et regroupe ses bâtiments.
Batiment	Organise les chambres appartenant à un bâtiment.
Chambre	Décrit une unité d'hébergement, notamment sa capacité et sa disponibilité.
DemandeHebergement	Conserve les informations relatives à une demande et à son état.
Attribution	Enregistre l'affectation d'une chambre à un étudiant pour une période donnée.

5. Modélisation UML

Les quatre diagrammes UML demandés décrivent les fonctionnalités du système, sa structure, le déroulement d'une attribution et les différents états d'une demande. Les diagrammes sont réalisés avec PlantUML. Pour conserver ce rapport simple, leurs images ne sont pas intégrées ici : elles sont indiquées comme disponibles dans le dépôt Git du projet.

5.1. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les acteurs externes et les services proposés par le système. Il permet de délimiter les responsabilités de l'étudiant, du responsable et de l'administrateur. Il met notamment en évidence la soumission d'une demande, la vérification des disponibilités, l'attribution d'une chambre et le suivi des occupations.

FIGURE 1 — DIAGRAMME DE CAS D'UTILISATION
Diagramme à consulter dans le dépôt Git du projet.

5.2. Diagramme de classes

Le diagramme de classes décrit la structure statique du système. Il présente les classes principales, leurs attributs, leurs opérations éventuelles et les associations qui les relient. Les classes retenues sont : Etudiant, Residence, Batiment, Chambre, DemandeHebergement et Attribution.

FIGURE 2 — DIAGRAMME DE CLASSES
Diagramme à consulter dans le dépôt Git du projet.

5.3. Diagramme de séquence : attribution d'une chambre

Le diagramme de séquence décrit les échanges chronologiques nécessaires à l'attribution. Le responsable sélectionne une demande. Le système vérifie son état, contrôle si l'étudiant possède déjà une chambre et consulte les places disponibles. Si une chambre adaptée est disponible, une attribution est créée, l'état de la chambre est mis à jour et la demande est acceptée. Sinon, la demande est refusée ou reste en attente selon la règle retenue.

FIGURE 3 — DIAGRAMME DE SÉQUENCE
Diagramme à consulter dans le dépôt Git du projet.

5.4. Diagramme d'états-transitions : demande d'hébergement

Ce diagramme représente le cycle de vie d'une demande. Après son enregistrement, elle prend l'état EnAttente. Elle peut devenir Acceptée lorsqu'une chambre est attribuée, Refusée si aucune place n'est disponible ou si le responsable refuse la demande, ou Annulée si l'étudiant renonce. Une demande acceptée peut devenir Terminée lorsque le départ est enregistré.

FIGURE 4 — DIAGRAMME D'ÉTATS-TRANSITIONS

Diagramme à consulter dans le dépôt Git du projet.

6. Règles de gestion principales

• Toute demande d'hébergement doit être associée à un étudiant identifié.

• Une demande nouvellement enregistrée prend l'état « EnAttente ».

• Une chambre ne peut être attribuée que si sa capacité permet d'accueillir l'étudiant.

• Avant toute attribution, le système vérifie la disponibilité de la chambre et l'absence d'une attribution incompatible.

• Une attribution conserve au minimum une date de début et, lorsque le départ est connu, une date de fin.

• Lorsqu'un départ est enregistré, la période d'occupation est clôturée et la disponibilité de la chambre est recalculée.

• L'historique des attributions est conservé afin de consulter les occupations passées.

7. Extension possible : statistiques d'occupation

Le système peut être amélioré par le calcul automatique du taux d'occupation de la résidence. Cet indicateur compare le nombre de places actuellement occupées au nombre total de places disponibles.

Taux d'occupation = (Nombre de places occupées ÷ Nombre total de places) × 100

Ces statistiques peuvent aider le responsable à suivre l'utilisation de la résidence, à repérer les périodes de forte occupation et à mieux planifier les futures attributions.

8. Outils envisagés

• PlantUML : réalisation des diagrammes UML à partir d'un langage textuel.

• C++ : langage prévu pour l'implémentation de l'application.

• Fichiers CSV ou texte : moyen de sauvegarde initiale des données.

Conclusion générale

Ce travail pratique a permis de présenter la conception d'un système de gestion d'une résidence universitaire à l'aide du langage UML. Le diagramme de cas d'utilisation définit les fonctionnalités et les acteurs. Le diagramme de classes décrit les données et les relations entre les objets. Le diagramme de séquence présente les échanges nécessaires à l'attribution d'une chambre, tandis que le diagramme d'états-transitions décrit l'évolution d'une demande d'hébergement.

Ces quatre représentations sont complémentaires et constituent une base pour le développement d'une future application. Une implémentation en C++, accompagnée d'une sauvegarde dans des fichiers, pourra être réalisée en respectant les règles de gestion définies.